

BUILDING BACKEND SYSTEMS

Week Two

Where Data Actually Lives

Six days of SQL — schema design, querying, joins, aggregation, subqueries, and normalization. The most important week in the curriculum.

WEEK	DAYS	TOOLS
Two of Six	8 – 14	PostgreSQL · psql

Contents

Day 8	Relational Databases from First Principles <i>Tables, constraints, primary keys, foreign keys, NULL</i>
Day 9	SQL: Reading Data <i>SELECT, WHERE, ORDER BY, LIMIT, CASE, column expressions</i>
Day 10	SQL: Combining Data with JOINS <i>INNER JOIN, LEFT JOIN, Cartesian products, anti-join</i>
Day 11	SQL: Aggregation and Grouping <i>COUNT, AVG, GROUP BY, HAVING, ranking patterns</i>
Day 12	SQL: Subqueries and Advanced Patterns <i>Subqueries, CTEs, window functions</i>
Day 13	Schema Design and Normalization <i>1NF, 2NF, 3NF, denormalization, Ocasia schema</i>
Day 14 — Summary	Week Two Consolidation <i>Review, diagnostics, reflection, and a clean GitHub push</i>

A NOTE ON THIS WEEK

Week Two is the most important week in the curriculum. Everything that follows — indexes, transactions, concurrency, ORMs, authentication, caching — assumes you understand relational databases at the level this week builds. If a concept does not click, stay on it. There is no penalty for taking two days on a single topic.

All SQL in this week runs in psql against a PostgreSQL database you install locally. No Python, no ORM, no abstraction. The goal is to see the database engine directly — as it receives a query, processes it, and returns results. That directness is what makes the later abstraction layers comprehensible.

Before you begin Day 8

Ensure PostgreSQL is installed and the postgres user password is set.

Verify you can connect: `psql -U postgres`

Create a folder: `mkdir week-02` in your repository.

Every SQL file you write this week goes in that folder.

Day 8

Relational Databases from First Principles

Week Two · Where Data Actually Lives

Everything you built in Week One disappeared the moment you stopped the server. A Python dictionary is not storage — it is memory, and memory is temporary. This week you will learn about permanent storage: relational databases. By the end of this week you will understand not just how to use a database, but why it is designed the way it is.

Today you install PostgreSQL, connect to it directly from the command line, and build your first real schema by hand. No ORM, no framework, no abstraction. Just SQL and the database engine. The goal is to understand the structure before you use the tools that hide it.

THEORY · 20 MINUTES

What a relational database actually is

A relational database is a system that stores data in tables, enforces rules about how that data is structured, and provides a language for querying and modifying it. The word relational comes from the mathematical concept of a relation — a set of tuples with named attributes. In practical terms it means: data is organised into tables, tables have defined columns, and relationships between tables are expressed through shared values rather than through nested objects or pointers.

This design was proposed by Edgar Codd in 1970 and remains the dominant model for structured data storage more than fifty years later. It survived because it solves a real problem: how do you store large amounts of structured data, query it efficiently, and update it consistently when multiple people are using it at the same time?

Tables, rows, columns, and schemas

A table is a collection of data about one type of thing. A students table contains data about students. A courses table contains data about courses. Each table has a fixed set of columns, where each column has a name and a data type. A row is one instance of that thing — one student, one course.

The schema is the structure: the definition of every table, every column, every constraint. The schema is separate from the data. You define the schema once, and the database enforces it every time you try to insert or update data. This enforcement is one of the core things a database gives you that a file system does not.

Primary keys

Every table needs a way to uniquely identify each row. A primary key is a column or set of columns whose value uniquely identifies a row in that table. No two rows can have the same primary key value. The primary key can never be null.

In practice you almost always use a surrogate key — an integer that has no business meaning, just exists to identify the row. PostgreSQL has a SERIAL type and a more modern GENERATED ALWAYS AS IDENTITY syntax that both auto-increment this integer for you. You will use GENERATED ALWAYS AS IDENTITY throughout this curriculum.

Foreign keys and referential integrity

A foreign key is a column in one table whose value references the primary key of another table. It is how you express a relationship: an enrollment row has a

`student_id` that references the `id` column of the `students` table. This is not just a convention — it is enforced by the database.

Referential integrity means the database will not let you create a foreign key value that points to a row that does not exist, and it will not let you delete a row that other rows are pointing to — unless you explicitly define what should happen in that case (`CASCADE`, `SET NULL`, `RESTRICT`). This enforcement is what makes a relational database more reliable than storing relationships in application code.

NULL and why it is dangerous

`NULL` means the absence of a value. It is not zero, not an empty string, not false — it is the absence of any value at all. `NULL` causes surprising behaviour: `NULL` compared to anything, including `NULL` itself, returns `NULL` rather than true or false. `NULL` in an aggregate function like `SUM` or `COUNT` is silently ignored. These behaviours cause bugs that are difficult to trace.

The practical rule: add `NOT NULL` to every column unless you have a specific reason why that column should be allowed to be absent. Fewer nullable columns means fewer unexpected behaviours.

PostgreSQL data types you will use most

Term	Meaning
<i>INTEGER</i>	A whole number. Use for IDs, counts, quantities.
<i>BIGINT</i>	A larger whole number. Use for IDs in high-volume tables or timestamps in milliseconds.
<i>NUMERIC(p,s)</i>	Exact decimal number with precision <i>p</i> and scale <i>s</i> . Use for money — never use <code>FLOAT</code> for financial values.
<i>TEXT</i>	Variable-length string with no size limit. Prefer over <code>VARCHAR</code> for most cases in PostgreSQL.
<i>VARCHAR(n)</i>	Variable-length string with maximum <i>n</i> characters. Use when you want the database to enforce a length limit.
<i>BOOLEAN</i>	True or false.

<i>TIMESTAMP WITH TIME ZONE</i>	Date and time with timezone. Always use the timezone-aware version in production.
<i>DATE</i>	A date with no time component.
<i>JSONB</i>	Binary JSON. Useful for semi-structured data. Indexed and queryable unlike TEXT-stored JSON.
<i>UUID</i>	A universally unique identifier. Useful as a public-facing ID when you do not want to expose sequential integers.

PRACTICAL · 60 MINUTES

Today you install PostgreSQL, connect to it, and build a real database schema by hand. No Python yet — just the database.

1. Install PostgreSQL and connect (10 minutes)

If you have not already installed PostgreSQL, download it from [postgresql.org](https://www.postgresql.org). During installation, note the password you set for the postgres user. Then connect from your terminal.

```
# Connect using psql — you will be prompted for the postgres
password
psql -U postgres

# Verify the connection — you should see the postgres=#
prompt
SELECT version();

# Create a database for this week's work
CREATE DATABASE university_db;

# Connect to it
\c university_db
```

All SQL in this curriculum runs in psql unless stated otherwise

2. Create your schema from scratch (25 minutes)

Write every CREATE TABLE statement by hand. Read every line before pressing Enter. The constraints are not optional — they are part of the lesson.


```

-- Students
CREATE TABLE students (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  name        TEXT          NOT NULL,
  email       TEXT          NOT NULL UNIQUE,
  year        INTEGER       NOT NULL CHECK (year BETWEEN 1
AND 4),
  gpa         NUMERIC(3,2) NOT NULL DEFAULT 0.00,
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Courses
CREATE TABLE courses (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  title       TEXT          NOT NULL,
  code        VARCHAR(10)  NOT NULL UNIQUE,
  credits     INTEGER       NOT NULL CHECK (credits BETWEEN 1
AND 6),
  capacity    INTEGER       NOT NULL DEFAULT 30
);

-- Enrollments: the junction table for the many-to-many
relationship
CREATE TABLE enrollments (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  student_id  INTEGER       NOT NULL REFERENCES students(id)
ON DELETE CASCADE,
  course_id   INTEGER       NOT NULL REFERENCES courses(id)
ON DELETE CASCADE,
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (student id, course id) -- a student cannot
enroll twice
);

```

Run each `CREATE TABLE` statement one at a time — read the output after each one

3. Insert real data (15 minutes)

Insert enough rows to make your queries meaningful later this week. Five students, four courses, and several enrollments.

```

INSERT INTO students (name, email, year, gpa) VALUES
  ('Amara Diallo',      'amara@uni.edu',    2, 3.45),

```

```

('Kwame Mensah',      'kwame@uni.edu',    3, 3.10),
('Fatima Al-Hassan', 'fatima@uni.edu',    1, 3.80),
('Kofi Acheampong',  'kofi@uni.edu',      4, 2.95),
('Ngozi Okonkwo',    'ngozi@uni.edu',     2, 3.60);

INSERT INTO courses (title, code, credits, capacity) VALUES
('Introduction to Databases',      'CS301', 3, 40),
('Software Engineering',           'CS302', 3, 35),
('Linear Algebra',                  'MA201', 3, 50),
('Computer Networks',              'CS401', 3, 30);

INSERT INTO enrollments (student_id, course_id) VALUES
(1,1), (1,2), (1,3),
(2,1), (2,4),
(3,1), (3,2), (3,3), (3,4),
(4,2), (4,4),
(5,1), (5,3);

```

You will use this data across all of Days 9, 10, 11, and 12

4. Test your constraints (10 minutes)

The constraints are only useful if you verify they work. Try each of these and observe what happens.

```

-- Try inserting a duplicate email - should fail
INSERT INTO students (name, email, year) VALUES ('Test',
'amara@uni.edu', 1);

-- Try inserting a year outside 1-4 - should fail
INSERT INTO students (name, email, year) VALUES ('Test',
'test@uni.edu', 5);

-- Try enrolling the same student in the same course twice -
should fail
INSERT INTO enrollments (student id, course id) VALUES (1,
1);

-- Try referencing a student that does not exist - should
fail
INSERT INTO enrollments (student_id, course_id) VALUES (999,
1);

-- Delete a student and check what happens to their
enrollments
DELETE FROM students WHERE id = 1;

```

```
SELECT * FROM enrollments WHERE student_id = 1;  
-- ON DELETE CASCADE means the enrollments are gone too
```

Re-insert student 1 after the delete test so your data is complete for Days 9–12

REFLECTION · 10 MINUTES

Answer these before moving on.

Question 1

What is the difference between a PRIMARY KEY constraint and a UNIQUE constraint? Can a table have multiple UNIQUE columns? Can it have multiple PRIMARY KEY columns?

Question 2

The enrollments table has ON DELETE CASCADE on both foreign keys. What would have happened if you had used ON DELETE RESTRICT instead? When would you choose RESTRICT over CASCADE?

Question 3

You used NUMERIC(3,2) for the GPA column rather than FLOAT. What is the difference between the two, and why does it matter for a value like a grade point average?

Question 4

You defined the schema before inserting any data. In a real project, what happens when you need to change the schema after data already exists? What is that process called?

BEFORE TOMORROW

Tomorrow you start writing SELECT queries against the data you just inserted. You will move from simple lookups to filtered queries to sorted and limited results. Come prepared — the most useful preparation is to look at your three tables and think about ten questions you could ask of this data.

Commit before you close your laptop

Save your SQL in week-02/day08_schema.sql

```
git add .
```

```
git commit -m "Day 8: university_db schema, constraints, referential integrity"
```


Day 9

SQL: Reading Data

Week Two · Where Data Actually Lives

SQL is the language of relational databases. It has been the standard for fifty years and is not going away. Today you write `SELECT` queries — the most important and most used operation in SQL. You will move from the simplest possible query to increasingly complex ones, building a mental model of how the database processes each request.

The goal is not to memorise syntax. The goal is to develop intuition for how the database thinks: what it scans, what it filters, what it sorts, and in what order. That intuition is what allows you to write queries that are not just correct but efficient.

THEORY · 20 MINUTES

How a `SELECT` query executes

SQL is a declarative language — you describe what you want, not how to get it. The database engine decides how to execute the query. But the logical order in which SQL clauses are processed is fixed, and understanding it is essential for writing correct queries.

```
-- Logical execution order (not the order you write them)
FROM          -- 1. Identify the table(s) to read from
```

```

WHERE      -- 2. Filter rows that do not match the condition
GROUP BY   -- 3. Group remaining rows by a column
HAVING     -- 4. Filter groups (not individual rows)
SELECT     -- 5. Choose which columns to return
ORDER BY   -- 6. Sort the result
LIMIT      -- 7. Return only N rows

```

Write queries in `SELECT...FROM...WHERE` order but think in `FROM...WHERE...SELECT` order

This order explains why you cannot use a column alias defined in `SELECT` inside a `WHERE` clause — the `WHERE` runs before `SELECT`, so the alias does not exist yet when `WHERE` is evaluated. It also explains why you can use a column alias in `ORDER BY` — `ORDER BY` runs after `SELECT`.

SELECT and column selection

`SELECT *` means return every column. It is convenient in development but a liability in production — it returns columns you may not need, breaks when columns are added or removed, and prevents the database from using certain optimisations. In application code, always name the columns you need.

WHERE and filtering

The `WHERE` clause filters rows. It supports comparison operators (`=`, `<>`, `<`, `>`, `<=`, `>=`), logical operators (`AND`, `OR`, `NOT`), range checks (`BETWEEN`), membership checks (`IN`), pattern matching (`LIKE`), and null checks (`IS NULL`, `IS NOT NULL`).

One operator deserves special attention: `LIKE`. The `%` wildcard matches any sequence of characters. The `_` wildcard matches exactly one character. `LIKE 'A%'` matches any value starting with `A`. `LIKE '%son'` matches any value ending with `son`. `LIKE` patterns are case-sensitive in PostgreSQL by default — use `ILIKE` for case-insensitive matching.

ORDER BY and LIMIT

`ORDER BY` sorts the result. Without an explicit `ORDER BY`, the database can return rows in any order — and that order can change between queries. Never rely on implicit ordering. If the order matters, always specify it.

LIMIT returns at most N rows. Always use ORDER BY together with LIMIT — without it, which N rows you get is undefined. OFFSET skips rows before returning results. OFFSET is how you implement pagination in SQL, though it has performance problems at large offsets that you will learn about in Week Three.

PRACTICAL · 60 MINUTES

Write all of the following queries against your `university_db`. Do not use an ORM or Python. Type every query in `psql`. The act of writing them by hand is how the syntax becomes intuitive.

1. Basic SELECT queries (15 minutes)

Start with the simplest possible queries and build up.

```
-- 1. Return all columns from students
SELECT * FROM students;

-- 2. Return only name and email
SELECT name, email FROM students;

-- 3. Return all students in year 2
SELECT name, year FROM students WHERE year = 2;

-- 4. Return students with a GPA above 3.3
SELECT name, gpa FROM students WHERE gpa > 3.3;

-- 5. Return students whose name starts with 'K'
SELECT name FROM students WHERE name LIKE 'K%';

-- 6. Return students in year 2 OR with GPA above 3.5
SELECT name, year, gpa FROM students WHERE year = 2 OR gpa > 3.5;

-- 7. Return students in year 2 AND with GPA above 3.3
SELECT name, year, gpa FROM students WHERE year = 2 AND gpa > 3.3;
```

week-02/day09_queries.sql — save every query you write today

2. Sorting and limiting (15 minutes)

These queries introduce ORDER BY and LIMIT. Think carefully about what happens when you use LIMIT without ORDER BY.

```
-- 8. All students sorted by GPA, highest first
SELECT name, gpa FROM students ORDER BY gpa DESC;

-- 9. The top 3 students by GPA
SELECT name, gpa FROM students ORDER BY gpa DESC LIMIT 3;

-- 10. Sort by year ASC, then GPA DESC within each year
SELECT name, year, gpa FROM students ORDER BY year ASC, gpa
DESC;

-- 11. The most recently enrolled student
SELECT name, enrolled_at FROM students ORDER BY enrolled_at
DESC LIMIT 1;

-- 12. Simulate page 2 of results (5 per page)
SELECT name FROM students ORDER BY id LIMIT 5 OFFSET 5;
```

Notice what happens when you run query 9 multiple times — is the result always the same?

3. Filtering with ranges, lists, and nulls (15 minutes)

These operators cover the remaining WHERE clause patterns you will use daily.

```
-- 13. Students in years 2 or 3 using IN
SELECT name, year FROM students WHERE year IN (2, 3);

-- 14. Students in years 2 or 3 using BETWEEN
SELECT name, year FROM students WHERE year BETWEEN 2 AND 3;

-- 15. Add a nullable column to students and test IS NULL
ALTER TABLE students ADD COLUMN phone TEXT;
UPDATE students SET phone = '+233501234567' WHERE id = 1;

SELECT name FROM students WHERE phone IS NULL;
SELECT name FROM students WHERE phone IS NOT NULL;

-- 16. Why NULL is dangerous — this returns 0 rows, not what
you expect
SELECT name FROM students WHERE phone = NULL;  -- WRONG: use
IS NULL
```

Query 16 is deliberate — `NULL = NULL` evaluates to `NULL`, not `TRUE`

4. Column expressions and aliases (15 minutes)

Queries can compute values and rename columns in the output.

```
-- 17. Concatenate name parts, uppercase the result
SELECT UPPER(name) AS full_name_upper, email FROM students;

-- 18. Compute how many years until graduation (assuming 4-
year degree)
SELECT name, year, (4 - year) AS years_remaining FROM
students;

-- 19. Show GPA as a percentage out of 100
SELECT name, ROUND((gpa / 4.0) * 100, 1) AS gpa_percent FROM
students;

-- 20. Use CASE to label GPA ranges
SELECT name, gpa,
       CASE
           WHEN gpa >= 3.5 THEN 'First Class'
           WHEN gpa >= 3.0 THEN 'Second Class Upper'
           WHEN gpa >= 2.5 THEN 'Second Class Lower'
           ELSE 'Pass'
       END AS classification
FROM students;
```

The CASE expression is SQL's if/else — you will use it frequently in reporting queries

REFLECTION · 10 MINUTES

These questions target the gaps that catch most beginners.

Question 1

Why does `SELECT * FROM students WHERE phone = NULL` return zero rows even when there are rows with NULL in the phone column? Write out what SQL is actually evaluating.

Question 2

You ran `LIMIT 3` without `ORDER BY`. Was the result the same every time you ran it? What guarantees (or does not guarantee) the order of rows when you do not specify `ORDER BY`?

Question 3

The logical execution order puts `SELECT` after `WHERE`. This means you cannot use a column alias from `SELECT` inside a `WHERE` clause. Write an example query that demonstrates this limitation and then rewrite it correctly.

Question 4

`BETWEEN` is inclusive on both ends. Write the equivalent query using `>=` and `<=` to confirm this. Then write a query that returns students with a GPA strictly between 3.0 and 3.5 — exclusive on both ends.

BEFORE TOMORROW

Today you read data from single tables. Tomorrow you combine data from multiple tables using `JOINS`. Before tomorrow, look at your enrollments table and ask yourself: if you want to see student names alongside the courses they are enrolled in, how would you do it? Think about what information each table has and what information it is missing.

Commit before you close your laptop

Save all 20 queries in `week-02/day09_queries.sql`

```
git commit -m "Day 9: SELECT queries, WHERE filtering, ORDER BY,
LIMIT, CASE expressions"
```


Day 10

SQL: Combining Data with JOINS

Week Two · Where Data Actually Lives

Yesterday you queried single tables. But the data you need is rarely in one table. To see a student's name alongside the courses they are enrolled in, you need to combine data from students, enrollments, and courses. JOINS are how you do this — and understanding what a JOIN actually does, rather than just memorising the syntax, is what allows you to write correct queries under pressure.

Today you will learn what a JOIN is at the mathematical level, understand the difference between join types, and write queries that span all three tables in your database. You will also deliberately write queries that break in interesting ways, because the bugs you create intentionally are the ones you understand most deeply.

THEORY · 20 MINUTES

What a JOIN actually is

A JOIN combines rows from two tables based on a condition. The naive mental model is that it merges two tables. The accurate mental model is that it first creates a Cartesian product — every possible combination of rows from both tables — and then filters that product down to only the rows where the join condition is true.

If the students table has 5 rows and the courses table has 4 rows, the Cartesian product has 20 rows — every student paired with every course. A JOIN with an ON clause then keeps only the pairs where the condition holds. Understanding this mental model explains why accidental JOINS — joining without an ON clause — can produce enormous and expensive result sets.

```
-- Accidental Cartesian product — do NOT run this on large
tables
SELECT * FROM students, courses; -- returns 5 × 4 = 20 rows

-- The correct version with an explicit JOIN condition
SELECT s.name, c.title
FROM   students s
JOIN   enrollments e ON e.student_id = s.id
JOIN   courses      c ON c.id          = e.course_id;
```

Always check your ON conditions — a missing one turns a 20-row query into a 20-million-row query

INNER JOIN vs LEFT JOIN vs RIGHT JOIN

An INNER JOIN returns only rows where the join condition is true in both tables. If a student has no enrollments, they do not appear in the result. If a course has no enrolled students, it does not appear.

A LEFT JOIN returns all rows from the left table, plus matching rows from the right table. Where there is no match, the right table's columns are NULL. This is how you find students with no enrollments: join students to enrollments with a LEFT JOIN, then filter for WHERE enrollments.id IS NULL.

A RIGHT JOIN is the mirror of a LEFT JOIN. It is rarely used — you can always rewrite a RIGHT JOIN as a LEFT JOIN by swapping the table order. Most engineers use only INNER JOIN and LEFT JOIN.

Table aliases and qualified column names

When you join multiple tables, the same column name can exist in both. Referring to id is ambiguous — is it students.id or courses.id? Always qualify

column names with a table alias when joining. Short aliases (s for students, c for courses, e for enrollments) make queries readable without being verbose.

PRACTICAL · 60 MINUTES

Write all queries in psql and save them to week-02/day10_joins.sql. Work through them in order — each builds on the last.

1. Basic INNER JOIN queries (20 minutes)

These queries combine two or three tables. Read each result carefully — verify it makes sense against the data you inserted on Day 8.

```
-- 1. Each student's name and the courses they are enrolled
in
SELECT s.name AS student, c.title AS course
FROM   students      s
JOIN   enrollments e ON e.student_id = s.id
JOIN   courses       c ON c.id = e.course_id
ORDER BY s.name, c.title;

-- 2. Only students enrolled in 'CS301'
SELECT s.name, s.email
FROM   students      s
JOIN   enrollments e ON e.student_id = s.id
JOIN   courses       c ON c.id = e.course_id
WHERE  c.code = 'CS301';

-- 3. All courses with the name and email of each enrolled
student
SELECT c.title, c.code, s.name, s.email
FROM   courses       c
JOIN   enrollments e ON e.course_id = c.id
JOIN   students      s ON s.id = e.student_id
ORDER BY c.title, s.name;

-- 4. Students enrolled in BOTH CS301 and CS302
SELECT s.name
FROM   students      s
JOIN   enrollments e1 ON e1.student_id = s.id
JOIN   courses       c1 ON c1.id = e1.course_id AND c1.code =
'CS301'
JOIN   enrollments e2 ON e2.student_id = s.id
```

```
JOIN    courses      c2 ON c2.id = e2.course_id AND c2.code =
'CS302';
```

week-02/day10_joins.sql

2. LEFT JOIN — finding missing relationships (20 minutes)

LEFT JOIN is how you answer questions about what is absent, not just what is present.

```
-- 5. All students, including those with no enrollments
SELECT s.name, e.course_id
FROM   students      s
LEFT JOIN enrollments e ON e.student_id = s.id
ORDER BY s.name;

-- 6. ONLY students with no enrollments
SELECT s.name
FROM   students      s
LEFT JOIN enrollments e ON e.student_id = s.id
WHERE  e.id IS NULL;

-- 7. All courses including those with no enrolled students
SELECT c.title, COUNT(e.id) AS enrolled_count
FROM   courses       c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title
ORDER BY enrolled_count DESC;

-- 8. Courses under capacity (enrolled < capacity)
SELECT c.title, c.capacity, COUNT(e.id) AS enrolled
FROM   courses       c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title, c.capacity
HAVING COUNT(e.id) < c.capacity
ORDER BY (c.capacity - COUNT(e.id)) DESC;
```

Query 6 is the classic anti-join pattern — LEFT JOIN + WHERE right side IS NULL

3. Break a JOIN intentionally (20 minutes)

These exercises produce wrong results deliberately. Understanding why they are wrong is more valuable than writing correct queries.

```
-- 9. Missing the ON clause — what does this produce?
SELECT s.name, c.title
```



```

FROM    students s, courses c;
-- Count the rows. Now you understand what a Cartesian
product is.

-- 10. Joining on the wrong column
SELECT s.name, c.title
FROM    students      s
JOIN    enrollments e ON e.student_id = s.year -- wrong
column
JOIN    courses       c ON c.id = e.course_id;
-- What happens? Why? What does the result mean?

-- 11. Using INNER JOIN where LEFT JOIN is needed
-- First add a student with no enrollments
INSERT INTO students (name, email, year) VALUES ('Yaw Darko',
'yaw@uni.edu', 1);

-- Now run INNER JOIN and LEFT JOIN and compare the row
counts
SELECT COUNT(*) FROM students s JOIN      enrollments e ON
e.student_id = s.id;
SELECT COUNT(*) FROM students s LEFT JOIN enrollments e ON
e.student_id = s.id;

```

Intentional breakage is how you develop instinct for debugging JOINS in production

REFLECTION · 10 MINUTES

Answer these before tomorrow.

Question 1

Explain what a Cartesian product is in one sentence without using SQL syntax. Then explain why accidentally producing one is dangerous in a production database.

Question 2

Query 6 uses LEFT JOIN + WHERE e.id IS NULL to find students with no enrollments. Could you write the same query using a subquery instead? Write it both ways and compare the readability.

Question 3

In query 7, you counted enrolled students with `COUNT(e.id)` rather than `COUNT(*)`. What is the difference? What would `COUNT(*)` return for a course with no enrollments, and why?

Question 4

You joined enrollments to courses in query 4 using two separate JOIN pairs. Could you solve the same problem with a subquery or a different approach? What are the tradeoffs?

BEFORE TOMORROW

You can now query across multiple tables. Tomorrow you aggregate that data — computing counts, averages, sums, and grouped summaries. This is where SQL starts to feel powerful: you can answer questions about your entire dataset in a single query rather than fetching all the rows and computing in Python.

Commit before you close your laptop

Save all 11 queries in `week-02/day10_joins.sql`

```
git commit -m "Day 10: INNER JOIN, LEFT JOIN, Cartesian products, anti-join pattern"
```


Day 11

SQL: Aggregation and Grouping

Week Two · Where Data Actually Lives

Aggregation is what makes SQL feel powerful. Rather than fetching every row and computing summaries in application code, you tell the database to compute them for you — count students per course, calculate average GPAs, find the highest enrollment, identify outliers. The database does this more efficiently than any application code can, because it processes the data where it lives.

Today you learn GROUP BY and the aggregate functions that work with it, the difference between WHERE and HAVING, and a mistake so common it deserves its own name — selecting a non-aggregated column without including it in GROUP BY.

THEORY · 20 MINUTES

Aggregate functions

An aggregate function takes a set of rows and returns a single value. The five standard aggregate functions cover the vast majority of use cases.

Term	Meaning
<code>COUNT(*)</code>	Counts all rows in the group, including NULLs.

<i>COUNT(col)</i>	Counts rows where col is not NULL. COUNT(*) and COUNT(col) can return different values when col has NULLs.
<i>SUM(col)</i>	Adds all non-NULL values in col.
<i>AVG(col)</i>	Returns the average of non-NULL values. AVG ignores NULLs, so it may not be what you expect.
<i>MIN(col)</i>	Returns the smallest non-NULL value.
<i>MAX(col)</i>	Returns the largest non-NULL value.

GROUP BY — what it does to rows

Without GROUP BY, an aggregate function treats the entire result set as one group and returns a single row. With GROUP BY, the result set is divided into groups — one group per distinct combination of the GROUP BY columns — and the aggregate function is applied to each group separately, returning one row per group.

The critical rule: every column in your SELECT clause must either appear in the GROUP BY clause or be wrapped in an aggregate function. Selecting a column that is neither is an error in standard SQL. PostgreSQL enforces this strictly.

```
-- ERROR: name is neither in GROUP BY nor aggregated
SELECT name, COUNT(*) FROM students GROUP BY year;

-- CORRECT: group by name too, or remove it
SELECT year, COUNT(*) AS count FROM students GROUP BY year;

-- ALSO CORRECT: aggregate name
SELECT year, COUNT(*) AS count, STRING_AGG(name, ', ') AS
names
FROM students
GROUP BY year;
```

PostgreSQL's error message for this mistake is explicit — read it carefully the first time you see it

WHERE versus HAVING

WHERE filters individual rows before grouping. HAVING filters groups after grouping. This distinction matters because you cannot use aggregate functions in a WHERE clause — the aggregation has not happened yet when WHERE runs.

```
-- WHERE filters rows BEFORE grouping
SELECT  year, AVG(gpa) AS avg_gpa
FROM    students
WHERE   year >= 2          -- exclude first-year students
      from the calculation
GROUP BY year;

-- HAVING filters groups AFTER grouping
SELECT  year, AVG(gpa) AS avg_gpa
FROM    students
GROUP BY year
HAVING  AVG(gpa) > 3.2;    -- only show years with average
      GPA above 3.2

-- ERROR: cannot use aggregate in WHERE
SELECT year FROM students WHERE AVG(gpa) > 3.2 GROUP BY year;
```

WHERE and HAVING are both filters — but they filter at different stages of execution

PRACTICAL · 60 MINUTES

Write all queries in psql and save them to `week-02/day11_aggregation.sql`. These queries answer real business questions about your university data.

1. Basic aggregation (15 minutes)

Start with whole-table aggregations, then add GROUP BY to break them down.

```
-- 1. Total number of students
SELECT COUNT(*) AS total_students FROM students;

-- 2. Average, min, and max GPA across all students
SELECT ROUND(AVG(gpa),2) AS avg_gpa, MIN(gpa) AS min_gpa,
      MAX(gpa) AS max_gpa
FROM    students;

-- 3. Number of students per year
```

```

SELECT year, COUNT(*) AS student_count
FROM   students
GROUP BY year
ORDER BY year;

-- 4. Average GPA per year
SELECT year, ROUND(AVG(gpa),2) AS avg_gpa, COUNT(*) AS
students
FROM   students
GROUP BY year
ORDER BY year;

-- 5. Number of enrollments per course
SELECT c.title, COUNT(e.id) AS enrolled
FROM   courses      c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title
ORDER BY enrolled DESC;

```

week-02/day11_aggregation.sql

2. HAVING and filtered groups (20 minutes)

These queries combine WHERE and HAVING to answer more specific questions.

```

-- 6. Years where average GPA is above 3.2
SELECT year, ROUND(AVG(gpa),2) AS avg_gpa
FROM   students
GROUP BY year
HAVING AVG(gpa) > 3.2;

-- 7. Courses with more than 2 enrolled students
SELECT c.title, COUNT(e.id) AS enrolled
FROM   courses      c
JOIN   enrollments e ON e.course id = c.id
GROUP BY c.id, c.title
HAVING COUNT(e.id) > 2;

-- 8. Students enrolled in more than 2 courses
SELECT s.name, COUNT(e.id) AS course_count
FROM   students      s
JOIN   enrollments e ON e.student_id = s.id
GROUP BY s.id, s.name
HAVING COUNT(e.id) > 2
ORDER BY course_count DESC;

```

```
-- 9. Year 2 students only – average GPA per year using WHERE
+ GROUP BY
SELECT year, ROUND(AVG(gpa), 2)
FROM   students
WHERE  year = 2
GROUP BY year;
```

Combine *WHERE* and *HAVING* in queries 9 – observe that *WHERE* runs before *GROUP BY*

3. Ranking and the most/least pattern (25 minutes)

A common class of queries asks for the top N or bottom N of a group. This is how you do it in SQL.

```
-- 10. The most enrolled course
SELECT c.title, COUNT(e.id) AS enrolled
FROM   courses      c
JOIN   enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title
ORDER BY enrolled DESC LIMIT 1;

-- 11. The student enrolled in the most courses
SELECT s.name, COUNT(e.id) AS course_count
FROM   students      s
JOIN   enrollments e ON e.student_id = s.id
GROUP BY s.id, s.name
ORDER BY course_count DESC LIMIT 1;

-- 12. Average number of courses per student
SELECT ROUND(AVG(course_count), 2) AS avg_courses_per_student
FROM   (
    SELECT student_id, COUNT(*) AS course_count
    FROM   enrollments
    GROUP BY student_id
  ) AS per_student;

-- 13. Courses where enrollment is at 75% capacity or above
SELECT c.title, c.capacity, COUNT(e.id) AS enrolled,
       ROUND(COUNT(e.id)::NUMERIC / c.capacity * 100, 1) AS
pct_full
FROM   courses      c
LEFT JOIN enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title, c.capacity
HAVING COUNT(e.id)::NUMERIC / c.capacity >= 0.75
ORDER BY pct_full DESC;
```


Query 12 uses a subquery in FROM — you will learn this more formally tomorrow

REFLECTION · 10 MINUTES

These questions target the conceptual gaps that aggregation commonly creates.

Question 1

In query 5 you used LEFT JOIN rather than INNER JOIN when counting enrollments per course. What would have happened if you used INNER JOIN? Which result is more useful for a course administrator and why?

Question 2

Query 12 uses a subquery in the FROM clause. Explain in plain English what that inner query computes and why it needs to be in a subquery rather than used directly.

Question 3

You used COUNT(e.id) rather than COUNT(*) in the JOIN queries. When do these produce different results? Write a small example that demonstrates the difference.

Question 4

Could you replace HAVING with a WHERE clause in any of today's queries? Which ones and how? What does this tell you about when each clause is necessary?

BEFORE TOMORROW

Tomorrow you go deeper into SELECT with subqueries and CTEs — tools that let you build complex queries out of simpler pieces. Before tomorrow, re-read

query 12 from today and make sure you understand why the inner query has to exist as a subquery. That intuition is the foundation for tomorrow.

Commit before you close your laptop

Save all 13 queries in week-02/day11_aggregation.sql

```
git commit -m "Day 11: aggregation, GROUP BY, HAVING, ranking  
patterns"
```


Day 12

SQL: Subqueries and Advanced Patterns

Week Two · Where Data Actually Lives

The queries you have written so far answer questions about data directly. Subqueries let you use the answer to one query as input to another — building complex logic from simple pieces. CTEs make that logic readable. Window functions let you compute aggregates without collapsing rows. These tools are what separate engineers who can write SQL from engineers who can write SQL well.

Today is the most technically demanding day of Week Two. Work slowly. Read every query result carefully before moving to the next one. The goal is not to write twenty queries — it is to understand five deeply.

THEORY · 20 MINUTES

What a subquery is

A subquery is a `SELECT` statement nested inside another SQL statement. It can appear in the `SELECT` clause (a scalar subquery returning one value), the `FROM` clause (a derived table), or the `WHERE` clause (a filter using the subquery's result). The database executes the innermost query first and uses its result in the outer query.

The distinction between correlated and uncorrelated subqueries matters for performance. An uncorrelated subquery executes once and its result is

reused. A correlated subquery references a column from the outer query and must re-execute for every row the outer query processes. Correlated subqueries can be extremely slow on large tables — recognising them is an important skill.

Common Table Expressions (CTEs)

A CTE is a subquery given a name and defined before the main query using the WITH keyword. It produces the same result as an equivalent subquery but is far more readable — especially when you need to reference the same derived data multiple times. Think of a CTE as giving a temporary name to a query result so you can refer to it by name in the query that follows.

```
-- Subquery version — harder to read
SELECT name FROM students
WHERE id IN (SELECT student_id FROM enrollments WHERE
course_id = 1);

-- CTE version — same result, easier to read and extend
WITH cs301_students AS (
    SELECT student_id FROM enrollments WHERE course_id = 1
)
SELECT s.name FROM students s
JOIN cs301_students cs ON cs.student_id = s.id;
```

CTEs become essential as queries grow — they are documentation as much as they are code

Window functions — aggregate without collapsing

An aggregate function with GROUP BY collapses many rows into one row per group. A window function computes an aggregate over a set of rows but keeps each row intact. You get the aggregate value as an additional column alongside the original row data. The OVER clause defines the window — which rows the function looks at for each row.

```
-- GROUP BY collapses to one row per year
SELECT year, AVG(gpa) FROM students GROUP BY year;

-- Window function: each student row keeps its data
```

```
-- PLUS the average GPA for their year appears alongside it
SELECT name, year, gpa,
       AVG(gpa) OVER (PARTITION BY year) AS year_avg_gpa
FROM   students;
```

Window functions are one of the most powerful tools in SQL — and one of the least used by beginners

PRACTICAL · 60 MINUTES

Save all queries in week-02/day12_advanced.sql. Read each result carefully before continuing.

1. Subqueries in WHERE (15 minutes)

These queries use subqueries to filter rows based on the result of another query.

```
-- 1. Students enrolled in CS301 — using a subquery
SELECT name FROM students
WHERE id IN (
    SELECT e.student_id
    FROM   enrollments e
    JOIN   courses      c ON c.id = e.course_id
    WHERE  c.code = 'CS301'
);

-- 2. Students with GPA above the overall average
SELECT name, gpa FROM students
WHERE  gpa > (SELECT AVG(gpa) FROM students)
ORDER BY gpa DESC;

-- 3. Students NOT enrolled in any course (subquery version
of the LEFT JOIN anti-join)
SELECT name FROM students
WHERE id NOT IN (SELECT DISTINCT student_id FROM
enrollments);

-- 4. Courses with more enrollments than the average course
enrollment
SELECT c.title, COUNT(e.id) AS enrolled
FROM   courses      c
JOIN   enrollments e ON e.course_id = c.id
GROUP BY c.id, c.title
```

```
HAVING COUNT(e.id) > (
    SELECT AVG(ec)
    FROM (SELECT COUNT(id) AS ec FROM enrollments GROUP
BY course_id) AS sub
);
```

week-02/day12_advanced.sql

2. CTEs for readable multi-step queries (20 minutes)

Rewrite query 4 from step 1 using a CTE, then build a more complex multi-CTE query.

```
-- 5. Query 4 rewritten with a CTE - compare readability
WITH enrollment_counts AS (
    SELECT course_id, COUNT(*) AS enrolled
    FROM enrollments
    GROUP BY course_id
),
avg_enrollment AS (
    SELECT AVG(enrolled) AS avg_enrolled FROM
enrollment_counts
)
SELECT c.title, ec.enrolled
FROM courses c
JOIN enrollment_counts ec ON ec.course_id = c.id
JOIN avg_enrollment ae ON ec.enrolled > ae.avg_enrolled;

-- 6. Full course report: title, enrolled, capacity,
pct_full, above/below avg
WITH counts AS (
    SELECT course_id, COUNT(*) AS enrolled FROM enrollments
GROUP BY course_id
),
overall_avg AS (
    SELECT AVG(enrolled) AS avg FROM counts
)
SELECT c.title,
    COALESCE(ct.enrolled, 0)
AS enrolled,
    c.capacity,

ROUND(COALESCE(ct.enrolled,0)::NUMERIC/c.capacity*100,1) AS
pct_full,
    CASE WHEN COALESCE(ct.enrolled,0) > oa.avg
```

```

        THEN 'Above average' ELSE 'Below average' END
AS vs_avg
FROM   courses      c
LEFT JOIN counts      ct ON ct.course_id = c.id
CROSS JOIN overall_avg oa
ORDER BY enrolled DESC;

```

Query 6 uses *COALESCE* to handle *NULL* from *LEFT JOIN* — *NULL* would break arithmetic

3. Window functions (25 minutes)

These queries keep individual rows while adding aggregate context — a capability *GROUP BY* cannot provide.

```

-- 7. Each student with their GPA and the average GPA for
their year
SELECT name, year, gpa,
       ROUND(AVG(gpa) OVER (PARTITION BY year), 2) AS
year_avg
FROM   students
ORDER BY year, gpa DESC;

-- 8. Rank students by GPA within their year (1 = highest)
SELECT name, year, gpa,
       RANK() OVER (PARTITION BY year ORDER BY gpa DESC) AS
rank_in_year
FROM   students;

-- 9. Each student's GPA versus the overall average
(deviation)
SELECT name, gpa,
       ROUND(gpa - AVG(gpa) OVER (), 2) AS
deviation_from_mean
FROM   students
ORDER BY deviation_from_mean DESC;

```

OVER() with no *PARTITION* means the entire result set is the window

REFLECTION · 10 MINUTES

These questions distinguish surface understanding from real comprehension.

Question 1

Explain the difference between a correlated and an uncorrelated subquery. Identify which of today's subqueries are correlated and which are uncorrelated. Why does the distinction matter for performance?

Question 2

Query 3 uses NOT IN with a subquery. There is a subtle NULL trap with NOT IN that can produce incorrect results. Research what happens when the subquery returns a NULL value and why IN and NOT IN behave differently.

Question 3

Compare query 4 (pure subquery) with query 5 (CTE version). Both produce the same result. When would you choose one over the other? What does the CTE version make easier?

Question 4

Query 8 uses RANK(). There is also DENSE_RANK() and ROW_NUMBER(). What is the difference between them? Write an example where the three functions would produce different results.

BEFORE TOMORROW

Tomorrow you step back from individual queries and design a real database schema from scratch. The challenge is not the SQL — you now know enough SQL. The challenge is the thinking: how do you model a domain as a set of tables and relationships that can support every query the application will ever need?

Commit before you close your laptop

Save all 9 queries in week-02/day12_advanced.sql

```
git commit -m "Day 12: subqueries, CTEs, window functions"
```


Day 13

Schema Design and Normalization

Week Two · Where Data Actually Lives

Knowing SQL lets you query a database. Knowing schema design lets you build one that does not break under load, resist modification, or produce subtly wrong query results. A poorly designed schema is a debt that compounds with every feature you add. A well-designed schema makes future queries almost write themselves.

Today you learn normalization — the formal theory behind good schema design — and then apply it by designing the full schema for a real product from scratch. The product is Ocasia, the study platform you have been building. You will design its database schema in the way a senior engineer would approach it: on paper first, with explicit reasoning for every decision.

THEORY · 20 MINUTES

What normalization is

Normalization is the process of structuring a relational database to reduce redundancy and improve data integrity. It is defined through a series of normal forms — each one stricter than the last. You need to understand the first three forms deeply. The higher forms exist but are rarely applied in practice.

The intuition behind normalization is simple: every piece of information should live in exactly one place. When the same information exists in multiple places, updating it requires changing multiple rows. Forgetting to update one of them creates an inconsistency. These inconsistencies are called anomalies.

First Normal Form (1NF)

A table is in 1NF if every cell contains exactly one atomic value — no lists, no arrays, no comma-separated values stored in a single column. Every row must be uniquely identifiable by a primary key.

```
-- Violates 1NF: courses stored as a comma-separated list
students
id | name           | courses
1  | Amara Diallo   | CS301, CS302, MA201

-- Correct: separate table for the relationship
enrollments
id | student_id | course_id
1  | 1          | 1
2  | 1          | 2
3  | 1          | 3
```

Storing lists in a single column makes every query that needs to use that list painful

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-key column depends on the entire primary key, not just part of it. This only applies to tables with composite primary keys — a single-column primary key makes 2NF violations impossible.

```
-- Violates 2NF: course_title depends on course_id alone, not
the full key
enrollments
student_id | course_id | course_title | enrolled_at
1          | 1         | Databases    | 2024-09-01
2          | 1         | Databases    | 2024-09-03
```

```
-- Correct: course_title belongs in the courses table, not
enrollments
enrollments: student_id, course_id, enrolled_at
courses:      id, title, code, credits
```

If you update the course title, you should change it in one row in one table — not fifty rows in enrollments

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and every non-key column depends directly on the primary key, not on another non-key column. A dependency between two non-key columns is called a transitive dependency.

```
-- Violates 3NF: department_head depends on department, not
on student_id
students
id | name   | department | department_head
1  | Amara  | CS         | Dr. Mensah
2  | Kofi   | CS         | Dr. Mensah
3  | Ngozi  | Business   | Prof. Boateng

-- Correct: department_head is a fact about department, lives
in departments table
students:      id, name, department_id
departments: id, name, department_head
```

When Dr. Mensah is replaced, one update in departments fixes all students — not a full table scan

When to denormalize

Normalization reduces redundancy and protects integrity. But it increases the number of JOINS required to answer queries, and JOINS have a cost. In read-heavy systems where query performance matters more than update simplicity, deliberately denormalizing — storing a calculated or duplicated value — is a valid engineering decision. The key word is deliberately. Denormalize with a reason, document the reason, and ensure you have a process for keeping the duplicated data consistent.

PRACTICAL · 60 MINUTES

You are going to design the database schema for Ocasia from scratch. Ocasia is an AI-powered study platform with students, instructors, courses, lessons, enrollments, progress tracking, and a Q&A system. Work on paper or in a text file first — do not open psql until you have a complete design.

1. Identify every entity (10 minutes)

An entity is a thing that your system needs to store information about.

List every entity in Ocasia and, for each one, list the attributes you need to store.

Entities to consider

User (a person who can be a student, instructor, or admin)

Course (a collection of lessons on a topic)

Lesson (a unit of content within a course — video, text, or exercise)

Enrollment (a student joining a course)

Progress (which lessons a student has completed)

Question (a student asking a question on a specific lesson)

Answer (an instructor or student answering a question)

2. Map the relationships (10 minutes)

For each pair of entities that are related, define the cardinality: one-to-one, one-to-many, or many-to-many. Many-to-many relationships require a junction table.

```
-- Write this out before coding:
```

```
User    <-> Course      : many-to-many (a user enrolls in many
courses; a course has many students)
Course  <-> Lesson      : one-to-many  (a course has many
lessons; a lesson belongs to one course)
User    <-> Lesson      : many-to-many (progress: a user
completes many lessons; a lesson is completed by many users)
Lesson  <-> Question    : one-to-many  (a lesson has many
questions)
```

```

Question <-> Answer      : one-to-many  (a question has many
answers)

-- Add anything you think is missing for your specific design

```

Drawing this as a diagram on paper is faster than writing it — but the writing forces precision

3. Write the full schema in SQL (30 minutes)

Now translate your design into CREATE TABLE statements. Apply 3NF. Add constraints. Choose data types carefully.

```

-- Save as week-02/day13_ocasia_schema.sql

CREATE TABLE users (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  name        TEXT          NOT NULL,
  email       TEXT          NOT NULL UNIQUE,
  password_hash TEXT        NOT NULL,
  role        TEXT          NOT NULL DEFAULT 'student'
              CHECK (role IN
('student','instructor','admin')),
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE TABLE courses (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  title       TEXT          NOT NULL,
  description TEXT,
  instructor_id INTEGER     NOT NULL REFERENCES users(id),
  published   BOOLEAN       NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE TABLE lessons (
  id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
  course_id   INTEGER       NOT NULL REFERENCES courses(id)
ON DELETE CASCADE,
  title       TEXT          NOT NULL,
  content     TEXT,
  position    INTEGER       NOT NULL, -- lesson order
              within the course

```



```

        UNIQUE (course_id, position)           -- no two lessons at
the same position
);

CREATE TABLE enrollments (
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    student_id  INTEGER          NOT NULL REFERENCES users(id)
ON DELETE CASCADE,
    course_id   INTEGER          NOT NULL REFERENCES courses(id)
ON DELETE CASCADE,
    enrolled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (student_id, course_id)
);

CREATE TABLE progress (
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    student_id  INTEGER          NOT NULL REFERENCES users(id)
ON DELETE CASCADE,
    lesson_id   INTEGER          NOT NULL REFERENCES lessons(id)
ON DELETE CASCADE,
    completed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (student_id, lesson_id) -- a lesson can only be
completed once
);

CREATE TABLE questions (
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    lesson_id   INTEGER          NOT NULL REFERENCES lessons(id)
ON DELETE CASCADE,
    author_id   INTEGER          NOT NULL REFERENCES users(id),
    body        TEXT             NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE answers (
    id          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    question_id INTEGER          NOT NULL REFERENCES
questions(id) ON DELETE CASCADE,
    author_id   INTEGER          NOT NULL REFERENCES users(id),
    body        TEXT             NOT NULL,
    is_accepted BOOLEAN          NOT NULL DEFAULT FALSE,

```

```
        created_at    TIMESTAMPTZ    NOT NULL DEFAULT NOW()
    );
```

week-02/day13_ocasia_schema.sql — run this in university_db or create a new ocasia_db

4. Review for normalization violations (10 minutes)

Read back through your schema and ask these specific questions about every table.

Normalization checklist

Does any column store multiple values? (1NF violation)

Does any non-key column depend on only part of a composite key? (2NF violation)

Does any non-key column depend on another non-key column rather than the primary key? (3NF violation)

Is there any information stored in two places? If yes, is there a good reason for the duplication?

Does every foreign key have a clear ON DELETE behaviour? Is it the right one?

REFLECTION · 10 MINUTES

These questions connect schema design to the application behaviour that depends on it.

Question 1

The progress table records when a lesson was completed but not how many times. A student might re-read a lesson. Should the schema track re-reads? What would you need to change to support that, and what are the tradeoffs?

Question 2

The lessons table has a position column to define lesson order within a course. What happens if an instructor wants to insert a new lesson between

position 2 and position 3? What query would you need to run, and does this design make that easy or hard?

Question 3

The users table has a role column with a CHECK constraint. An alternative design would be to have separate tables for students and instructors. What are the tradeoffs between a single users table with a role versus separate tables per role?

Question 4

You chose ON DELETE CASCADE for most foreign keys. For which relationships might ON DELETE RESTRICT be more appropriate, and why?

BEFORE TOMORROW

Tomorrow is the Week Two summary day. Before you go to sleep tonight, run your Ocasia schema in psql, insert a few rows of test data, and write two or three queries against it. This is the final piece of consolidation before the summary.

Commit before you close your laptop

Save your schema in week-02/day13_ocasia_schema.sql

```
git commit -m "Day 13: normalization, Ocasia schema design"
```


Week 2 Summary

Consolidation and Reflection

No new concept today. This is the most important day of the week — not because of what you will learn, but because of what you will consolidate. The database week is the foundation that every subsequent week builds on. The concepts you solidify today will carry you through transactions, indexing, concurrency, ORMs, and eventually the KasiCredit capstone.

Work slowly. Every item in this summary day has a specific purpose. The queries you write today are not exercises — they are diagnostics. They will show you exactly where your understanding is solid and exactly where it is not.

THE WEEKLY SUMMARY

Write five to ten sentences answering this question: if a product manager asked you to build a report showing each student's name, the number of courses they are enrolled in, their average GPA, and a label showing whether they are above or below the overall average GPA — how would you write that query?

Do not just describe the query in words. Write the SQL. Then run it and verify the result. If you cannot write it without looking at your notes, revisit the specific day that covers the concept you are struggling with before moving to Week Three.

Write the query before reading any further

This is the diagnostic. If you can write it correctly, Week Two is solid.

If you cannot, identify exactly which clause is blocking you — that is today's work.

Save the query in week-02/SUMMARY.sql

REVIEW CHECKLIST

Work through each item. If any answer is no, go back to the relevant day before Week Three begins.

1. Can you explain what a primary key is and why every table needs one?
2. Can you explain what a foreign key is and what referential integrity means — in plain English, without using the words 'foreign' or 'key' in your explanation?
3. Can you write a query using INNER JOIN, LEFT JOIN, and GROUP BY from memory — without looking at notes?
4. Can you explain the difference between WHERE and HAVING and give a concrete example of when each is needed?
5. Can you explain what 1NF, 2NF, and 3NF mean and identify a violation of each in a table someone shows you?
6. Can you write a CTE that makes a complex query readable?
7. Can you name three specific things that the database engine enforces for you that application code cannot reliably enforce?

SQL CONSOLIDATION QUERIES

Write these five queries from memory. Do not look at Days 9 through 12 until after you have attempted each one. Time yourself — each should take under five minutes if the concepts are solid.

Query A — Aggregation with filtering

For each year, show the number of students and the average GPA, but only for years that have more than one student.

Query B — Multi-table join

Show the full name of every student, the title of every course they are enrolled in, and the course code — ordered alphabetically by student name.

Query C — Anti-join

Show the names of all students who are NOT enrolled in the course with code 'CS301'. Write it two ways: once using LEFT JOIN and once using NOT IN.

Query D — Window function

For each enrollment, show the student's name, the course title, the date of enrollment, and the student's rank by enrollment date within that course — earliest enrollment ranked 1.

Query E — Schema design

Write a CREATE TABLE statement for a table called payments with columns for: a primary key, a student reference, an amount in a currency-safe format, a payment method (card, mobile money, bank transfer), a status (pending, completed, failed), and a timestamp. Include all appropriate constraints.

SCHEMA REVIEW

Open your Ocasia schema from Day 13. Look at every table and ask yourself three questions.

For each table in your Ocasia schema

What query would break if I dropped this table? That tells you its role in the system.

What happens to related data if I delete a row from this table? Is that the right behaviour?

Is there any information that is stored in more than one place? If so, what process keeps them consistent?

Make any corrections you find. A schema that you have reviewed and can defend is more valuable than a perfect schema you cannot explain.

REFLECTION QUESTIONS

Answer these in writing — the act of writing forces precision that mental answers do not.

Looking back at Week Two

Which SQL concept took the longest to feel natural — and what finally made it click?

What is one mistake you made in a query this week that you will not make again, and why?

If you had to explain a LEFT JOIN to someone who has never written SQL, what analogy would you use?

Looking forward to Week Three

Week Three is about making databases fast and safe: indexes, transactions, concurrency, and ORMs.

You have written queries against a database with five students and four courses.

What do you think would happen to your query performance if the students table had 500,000 rows?

What would you need to change to make queries fast at that scale? Write down your hypothesis before Day 15.

FINAL COMMIT

Before you close your laptop, your repository should contain:

8. week-02/day08_schema.sql — CREATE TABLE statements and constraint tests
9. week-02/day09_queries.sql — 20 SELECT queries from basic to CASE expressions
10. week-02/day10_joins.sql — INNER JOIN and LEFT JOIN queries including intentional breakage
11. week-02/day11_aggregation.sql — GROUP BY, HAVING, and ranking queries
12. week-02/day12_advanced.sql — subqueries, CTEs, and window functions
13. week-02/day13_ocasia_schema.sql — full Ocasia schema with all tables and constraints
14. week-02/SUMMARY.sql — the diagnostic query and your five consolidation queries

```
git add .
git commit -m "Week 2 complete: relational databases, SQL,
schema design"
git push
```

Your final commit for Week Two

WEEK THREE PREVIEW

Week Three is where database work becomes systems engineering. You will create a table with half a million rows and measure the difference between a query that scans every row and one that uses an index — a difference that can be the gap between milliseconds and minutes. You will simulate a bank transfer to understand why transactions exist. You will send a hundred simultaneous requests to a counter endpoint and watch it produce the wrong answer, then fix it.

You will also connect your FastAPI application to a real PostgreSQL database for the first time — replacing the in-memory dictionaries you have been using since Day 1. The application you have been building piece by piece will start to look like a real system.

The database week was about understanding what databases are. Week Three is about understanding how to use them correctly under real conditions. The two weeks together are the foundation that every week after them rests on.